

Primer Parcial de Programación Orientada a Objetos (72.33)

16/04/2026

Ejercicio 1 (3 puntos)

Se desea modelar un iterador para vectores genéricos que recibe:

- **Un vector**
- **Una función para transformar cada uno de los elementos del vector**, de forma que se termina iterando por el resultado de aplicar dicha función a cada uno de los elementos del vector (`java.util.function.Function`);
- **Un predicado que modela una condición que se debe evaluar para cada uno de los elementos del vector** y registra la cantidad de elementos ya retornados por `next()` que cumplieron con el predicado (`java.util.function.Predicate`).

Implementar `MatchCountingIterator` y todo lo necesario para que, con el siguiente programa de prueba, se obtenga **la salida indicada en los comentarios**:

```
void main() {
    // 1. Se instancia un iterador que recibe
    // - Un vector de Integer
    // - Una función que convierte de Integer a String usando dos dígitos
    // - Un predicado de Integer que indica si el entero es par
    Integer[] intValues = {2, 3, 1, 8, 4};
    Function<Integer, String> intStrFn = n → "%02d".formatted(n);
    Predicate<Integer> intPr = n → n % 2 == 0;
    MatchCountingIterator<Integer, String> intStrIt =
        new MatchCountingIterator<>(intValues, intStrFn, intPr);
    IO.println(intStrIt.getMatchCount()); // 0
    while (intStrIt.hasNext()) {
        String elem = intStrIt.next();
        int matchCount = intStrIt.getMatchCount();
        IO.println("%s < %d".formatted(elem, matchCount));
    }
    // 02 < 1
    // 03 < 1
    // 01 < 1
    // 08 < 2
    // 04 < 3
    try {
        intStrIt.next();
    } catch (Exception ex) {
        IO.println("Error"); // Error
    }

    // 2. Se instancia un iterador que recibe
    // - Un vector de String
    // - Una función que convierte de String a Integer usando la longitud
    // - Un predicado de String que indica si empieza con 'a'
    String[] stringValues = {"hola", "a", "adios", "mundo"};
    Function<String, Integer> strIntFn = w → w.length();
    Predicate<String> strPr = w → w.startsWith("a");
    MatchCountingIterator<String, Integer> strIntIt =
        new MatchCountingIterator<>(stringValues, strIntFn, strPr);
    IO.println(strIntIt.getMatchCount()); // 0
    while (strIntIt.hasNext()) {
        Integer elem = strIntIt.next();
        int matchCount = strIntIt.getMatchCount();
        IO.println("%s < %d".formatted(elem, matchCount));
    }
}
```

```
}  
// 4 < 0  
// 1 < 1  
// 5 < 2  
// 5 < 2  
}
```

Ejercicio 2 (3.5 puntos)

Se desea modelar un sistema para **registrar el despacho de equipajes** en una estación de check-in, **indicando los detalles del equipaje despachado con cierto orden**.

Ya cuenta con la clase `Luggage` que modela a un equipaje despachado (a partir de un “tag” o etiqueta (String) y el peso en kg (int) y puede modificarse:

```
public class Luggage {  
    private final String tag;  
    private final int weight;  
  
    public Luggage(String tag, int weight) {  
        this.tag = tag;  
        this.weight = weight;  
    }  
  
    public String getTag() {  
        return tag;  
    }  
  
    public int getWeight() {  
        return weight;  
    }  
}
```

Al instanciarse, **se indica por parámetro el peso en kg correspondiente a la franquicia**. Así, todos los equipajes despachados que tengan un peso mayor al de la franquicia serán marcados como equipaje en exceso (“EXCESS”), el resto será aceptado (“OK”). Esta franquicia debe poder ser actualizada en cualquier momento con el método `setAllowance`. Para simplificar el ejercicio, no se debe validar que la misma es un valor positivo.

Se debe ofrecer un método `checkIn` para despachar un equipaje, indicando el *tag* y el peso.

Se deben contar con tres métodos para consultar una copia del equipaje despachado hasta el momento:

- `luggages()` en el orden en que se despacharon
- `luggagesByWeightDesc()` en orden descendente por peso y desempata alfabético por tag
- `luggagesByTagAsc()` en orden alfabético por tag y desempata descendente por peso

En los tres, además de indicar el *tag* y el peso de cada equipaje despachado, se debe indicar la franquicia del sistema y si el equipaje está en exceso o no.

Implementar `FlightLuggage` y todo lo necesario para que, con el siguiente programa de prueba, **se obtenga la salida indicada en los comentarios:**

```
void main() {  
    // Se instancia un sistema para despachar equipaje  
    // donde la franquicia es de 23kg.  
    FlightLuggage fL = new FlightLuggage(23);
```

```

// Se despacha una valija con el tag "ABC123" de 20 kg
fL.checkIn("ABC123", 20);
fL.checkIn("XYZ987", 23);
fL.checkIn("ABC123", 25);
fL.checkIn("AAA000", 19);
fL.checkIn("ZZZ999", 23);

// Se indica, para cada equipaje despachado, el tag, el peso en kg,
// si está dentro de la franquicia y el valor en kg de la franquicia
// en el orden en que se despacharon
for (Luggage luggage : fL.luggages()) {
    IO.println(luggage);
}
// Luggage ABC123 ◊ 20 kg ◊ OK (Allowance: 23 kg)
// Luggage XYZ987 ◊ 23 kg ◊ OK (Allowance: 23 kg)
// Luggage ABC123 ◊ 25 kg ◊ EXCESS (Allowance: 23 kg)
// Luggage AAA000 ◊ 19 kg ◊ OK (Allowance: 23 kg)
// Luggage ZZZ999 ◊ 23 kg ◊ OK (Allowance: 23 kg)

fL.setAllowance(20);

// Se indica, para cada equipaje despachado, el tag, el peso en kg,
// si está dentro de la franquicia y el valor en kg de la franquicia
// en orden descendente por peso y desempata alfabético por tag
for (Luggage luggage : fL.luggagesByWeightDesc()) {
    IO.println(luggage);
}
// Luggage ABC123 ◊ 25 kg ◊ EXCESS (Allowance: 20 kg)
// Luggage XYZ987 ◊ 23 kg ◊ EXCESS (Allowance: 20 kg)
// Luggage ZZZ999 ◊ 23 kg ◊ EXCESS (Allowance: 20 kg)
// Luggage ABC123 ◊ 20 kg ◊ OK (Allowance: 20 kg)
// Luggage AAA000 ◊ 19 kg ◊ OK (Allowance: 20 kg)

// Se indica, para cada equipaje despachado, el tag, el peso en kg,
// si está dentro de la franquicia y el valor en kg de la franquicia
// en orden alfabético por tag y desempata descendente por peso
int allowance = 24;
for (Luggage luggage : fL.luggagesByTagAsc()) {
    fL.setAllowance(allowance--);
    IO.println(luggage);
}
// Luggage AAA000 ◊ 19 kg ◊ OK (Allowance: 24 kg)
// Luggage ABC123 ◊ 25 kg ◊ EXCESS (Allowance: 23 kg)
// Luggage ABC123 ◊ 20 kg ◊ OK (Allowance: 22 kg)
// Luggage XYZ987 ◊ 23 kg ◊ EXCESS (Allowance: 21 kg)
// Luggage ZZZ999 ◊ 23 kg ◊ EXCESS (Allowance: 20 kg)
}

```

Ejercicio 3 (3.5 puntos)

Se desea modelar un sistema para la **gestión de depósitos** de utilería de una productora audiovisual contemplando los depósitos de cajones (*cranes*).

Ya cuenta con la clase **DepositOperation** (que puede modificarse), que modela una operación de guardado en el depósito y consiste en una descripción (`String`), un peso en kg (`int`) y la hora y minutos de la operación (`java.time.LocalDateTime`).

```

public class DepositOperation {
    private final String description;
    private final int weight;
    private final LocalTime localTime;
    public DepositOperation(String description, int weight, LocalTime
localTime) {
        this.description = description;
        this.weight = weight;
        this.localTime = localTime;
    }
    public String getDescription() {
        return description;
    }
    public int getWeight() {
        return weight;
    }
    public LocalTime getLocalTime() {
        return localTime;
    }
}

```

Para instanciar un depósito se indican los **horarios de apertura y cierre** (`java.time.LocalTime`) y el **peso máximo en kg (int)** permitido para un cajón. Todos los depósitos deben contar con un método **store** que recibe una operación (`DepositOperation`) y un método **getCrates()** que indica la cantidad de operaciones que se realizaron con éxito.

En caso de invocar a `store()` con una operación que esté fuera de los horarios de operación del depósito o que tenga un peso que no es positivo o un peso que es mayor al permitido **se debe arrojar un error (`CannotStoreException`)**.

Existen tres tipos de depósitos:

- **Uno que permite depósitos respetando las restricciones ya mencionadas**
- **Uno que permite depósitos respetando las restricciones ya mencionadas donde además el peso del cajón debe ser mayor o igual al del último depositado con éxito (salvo que sea el primero)**
- **Uno que permite depósitos respetando las restricciones ya mencionadas donde además el peso del cajón debe ser mayor o igual al del último depositado con éxito (salvo que sea el primero) y transcurra al menos una cantidad de minutos (que se recibe por parámetro) entre la operación y la del último depositado con éxito (salvo que sea el primero)**

Para simplificar el ejercicio no corresponde verificar que el horario de apertura es anterior al horario de cierre.

Completar losA.....B.....C....., implementar todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```

void main() {
    // Ejemplo de uso de la clase java.time.LocalTime
    IO.println(LocalTime.of(18, 10).isAfter(LocalTime.of(17, 50))); // true
    IO.println(LocalTime.of(18, 10).isBefore(LocalTime.of(17, 50))); // false
    IO.println(LocalTime.of(18, 10).plusMinutes(60)); // 19:10

    DepositOperation throne =
        new DepositOperation("Royal Throne", 40, LocalTime.of(9, 0));
    DepositOperation armor =
        new DepositOperation("Knight Armor", 55, LocalTime.of(10, 15));
    DepositOperation torch =
        new DepositOperation("Medieval Torch", 55, LocalTime.of(10, 20));
}

```

```

DepositOperation banner =
    new DepositOperation("Castle Banner", 12, LocalTime.of(10, 30));

// Se instancia un depósito que opera entre las 08:00 y las 13:00 inclusive
// que admite cajones de hasta 60 kg
..... depot = new .....(
    LocalTime.of(8, 0), LocalTime.of(13, 0), 60);
// Se guarda el cajón Royal Throne
depot.store(throne);
IO.println(depot.getCrates()); // 1
depot.store(armor);
IO.println(depot.getCrates()); // 2
try { // Falla porque el peso no es positivo
    depot.store(new DepositOperation("Fake", 0, LocalTime.of(10, 35)));
} catch (CannotStoreException ex) {
    IO.println(ex.getMessage()); // Cannot store crate in depot
}
try { // Falla porque la hora está fuera del rango permitido
    depot.store(new DepositOperation("Door", 50, LocalTime.of(14, 0)));
} catch (CannotStoreException ex) {
    IO.println(ex.getMessage()); // Cannot store crate in depot
}

// Se instancia un depósito que opera entre las 08:00 y las 13:00 inclusive
// que admite cajones de hasta 60 kg donde el peso del cajón debe ser
// mayor o igual al último depositado con éxito (salvo que sea el primero)
..... incrWeightDepot = new .....(
    LocalTime.of(8, 0), LocalTime.of(13, 0), 60);
incrWeightDepot.store(throne);
IO.println(incrWeightDepot.getCrates()); // 1
incrWeightDepot.store(armor);
IO.println(incrWeightDepot.getCrates()); // 2
incrWeightDepot.store(torch);
IO.println(incrWeightDepot.getCrates()); // 3
// Falla porque 12 es menor que el peso del último depositado con éxito
try {
    incrWeightDepot.store(banner);
} catch (CannotStoreException ex) {
    IO.println(ex.getMessage()); // Cannot store crate in depot
}

// Se instancia un depósito que opera entre las 08:00 y las 13:00 inclusive
// que admite cajones de hasta 60 kg donde el peso del cajón debe ser
// mayor o igual al último depositado con éxito (salvo que sea el primero)
// y donde deben pasar al menos 15 minutos desde el último depositado con
// éxito (salvo que sea el primero)
..... timeSpacedDepot = new .....(
    LocalTime.of(8, 0), LocalTime.of(13, 0), 60, 15);
timeSpacedDepot.store(throne);
IO.println(timeSpacedDepot.getCrates()); // 1
timeSpacedDepot.store(armor);
IO.println(timeSpacedDepot.getCrates()); // 2
// Falla porque no pasaron 15 minutos desde el último depositado con éxito
try {
    timeSpacedDepot.store(torch);
} catch (CannotStoreException ex) {
    IO.println(ex.getMessage()); // Cannot store crate in depot
}
}

```